Predicting Spotify Streaming Success Using Audio Features and Platform Exposure Metrics

Matthew Zhou

December 2025

1 Introduction

Streaming counts on Spotify are a widely used measure of a song's reach and commercial success. Being able to anticipate how strongly a track will perform is useful for decisions such as promotional strategy, playlist placement, and understanding which song attributes are associated with higher audience engagement. Streaming performance is influenced by a mix of factors and data through many audio features (e.g., tempo, energy, and danceability) and platform exposure features (e.g., counts of playlist and chart appearances across Spotify and other services). In this project, the goal is to predict a song's Spotify streaming performance from a combination of numerical audio features, platform exposure metrics, and categorical musical attributes available in the dataset. To answer this prediction problem, we need to fit both linear and nonlinear machine learning models that combine the different streaming feature types, and we compare their out-of-sample performance to determine which approach generalizes best. The goal is to identify the most accurate model and to understand which features are most predictive of streaming success.

1.1 Codebase and Dataset Links

Code:`https://colab.research.google.com/drive/1eXX5pjDzJIiG_NZnR9JXJ_ELPS3inLCT?usp=sharing`
Dataset: `https://www.kaggle.com/datasets/abdulszz/spotify-most-streamed-songs`

2 Data

2.1 Dataset and Target Variable

We used a dataset of popular Spotify songs from Kaggle, where each row corresponds to a unique track (`track_name`) and columns contain artist metadata (`artist(s)_name`, `artist_count`), release dates (`released_year`, `released_month`, `released_day`), platform exposure metrics, and audio features. Platform exposure variables include counts of playlist and chart appearances across Spotify, Apple Music, Deezer, and Shazam (e.g., `in_spotify_playlists`, `in_spotify_charts`, `in_apple_playlists`, etc.). Audio/metadata variables include: `bpm`, `danceability_%`, `energy_%`, `valence_%`, `acousticness_%`, `instrumentalness_%`, `liveness_%`, and `speechiness_%`, as well as categorical features `key` and `mode`.

Our outcome variable is Spotify `streams`. Since stream counts are highly right-skewed, we model the transformed response

$$\texttt{log\_streams} = \log(1 + \texttt{streams})$$

in order to compresses extreme values and improves model stability.[1]

2.2 Preprocessing and Modeling Setup

To ensure features are consistently formatted across all models, we first removed duplicate rows and cast release date variables `released_year`, `released_month`, and `released_day` as integers. We then cleaned numeric columns that were stored as strings by converting values to strings, removing commas and spaces,

replacing dashes with 0, and changing types to numeric. Any remaining missing values in those columns were then filled with 0. For the categorical feature `key`, missing values were filled with the explicit category `Unknown`. Once the data was cleaned, we constructed the response variable `log_streams` and defined the predictor matrix as a mix of numerical and categorical features. To ensure consistent preprocessing across all models, we used a single scikit-learn `Pipeline` with a `ColumnTransformer`: numeric features are standardized using `StandardScaler`, and categorical features are converted to indicator variables using `OneHotEncoder()`. Standardization is important for models that depend on feature scale (linear regression and SVR), and one-hot encoding allows categorical variables to be used in regression models.

We split the dataset into training and test sets using an 80/20 split. All models are trained on the same training set and evaluated on the same held-out test set, so that later performance comparisons are fair and reflect out-of-sample generalization.

3 Analytical Models

3.1 Evaluation Metrics

We evaluate predictive performance on the held-out test set using RMSE, MAE, and R^2. RMSE and MAE summarize prediction error magnitude (with RMSE penalizing large errors more), and test-set R^2 reports explained variance relative to a mean baseline.

3.2 Linear and Regression Models

We fit a Multiple Linear Regression model in order to predict *log_streams* by using standardized numerical features and one-hot encoded categorical variables. Doing so will provide us with a model that is simple and has interpretable baseline for comparison with more advanced machine learning methods/approaches.

Since many predictors in our dataset are correlated with each other, we used regularized linear models, like Ridge, Lasso, and Elastic Net regression. These regressions were used to reduce multicollinearity and improve the stability of our model by shrinking the coefficients. When we examined multicollinearity among numeric features, we calculated the variance VIF for our predictors, and found that most of them had moderate VIF values, suggesting that multicollinearity did exist, but was not as extreme (Except for one).[2]

All of these models were training using the same preprocessing pipeline and evaluated on a test set. We saw that regularization provided some improvements over our Multiple Linear Regression model, suggesting that just a linear structure is not enough to capture the complexity of streaming performances. However, despite this, we did see that variables related to playlists continued to be one of the strongest predictors in our models.

3.3 Nonlinear Models: Random Forest, SVR, XGBoost

In order to evaluate nonlinear relationships in platform exposure data, audio features, and streaming performance, we used three advanced models: Random Forest, Support Vector Regression, and XGBoost. All models were implemented in a pipeline for consistent pre-processing.

Each model underwent hyperparamter optimization using 5-fold GridSearchCV, with RMSE being used as our target metric. The models were retrained on the training dataset and evaluated on the held-out test set, with an 80-20 split. We assessed RMSE, MAE, and R^2 to judge the predictive performance and error magnitude of each model. The consistent tuning and evaluation let us compare models under similar conditions, allowing us to more easily identify the best approach to unseen data.

The Random Forest model had the strongest results, with the lowest RMSE (0.516), lowest MAE (0.403), and highest R^2 (0.736). The Random Forest model showed the best generalization and balance in bias and variance for nonlinear models. XGBoost also performed relatively well, but had slightly worse error metrics, suggesting it could have benefited from more data or tuning. Nonlinear models consistently outperformed the linear models, showing the presence of nonlinear relationships in the data.

4 Results

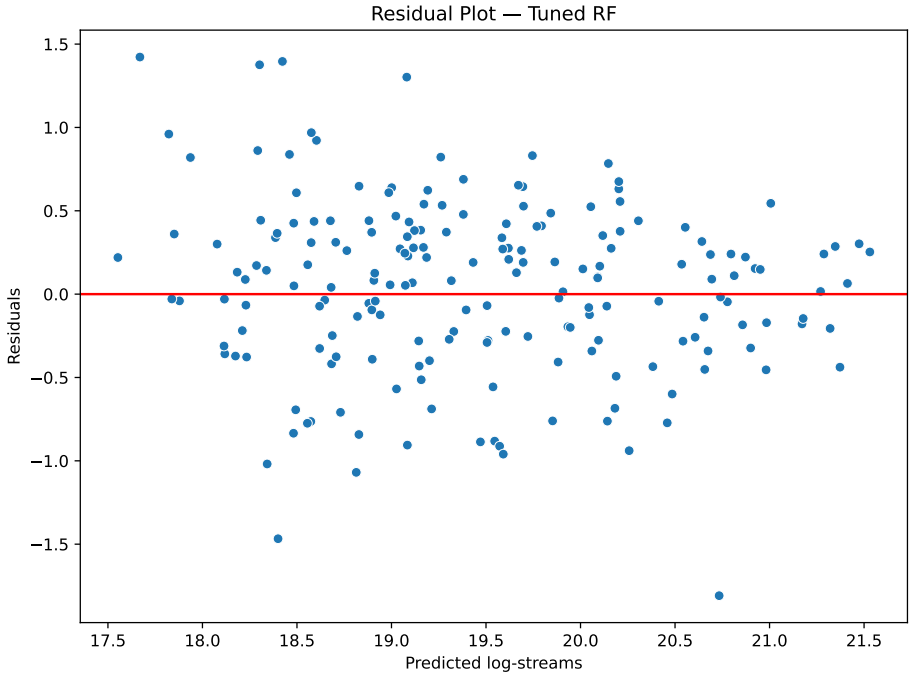
4.1 Performance Comparison and Model Selection

Model Performance Comparison

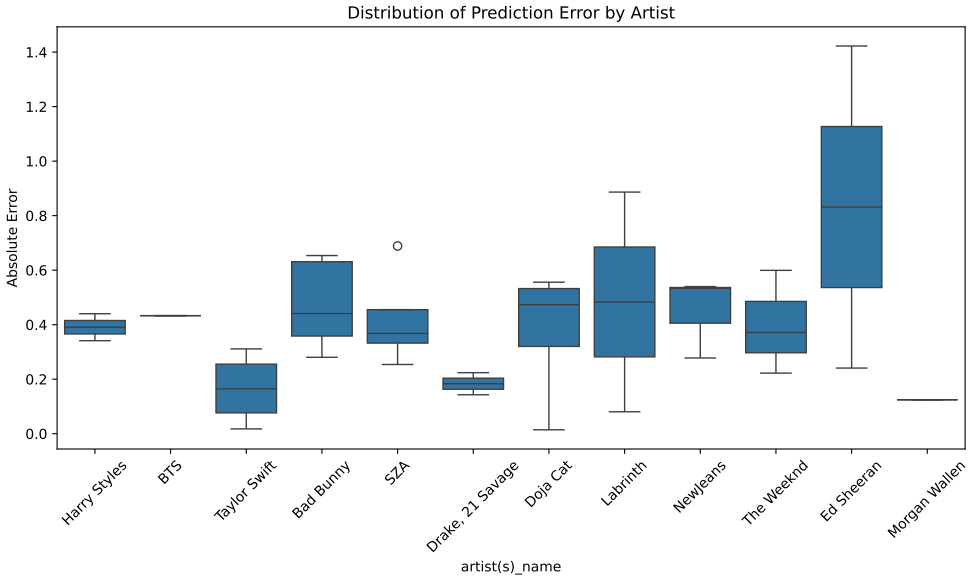
Model	RMSE	MAE	R^2
Linear Regression	0.7227467089161385	0.5773962727762589	0.4825566350639965
Elastic Net	0.7185394541430932	0.5886598561250014	0.488563385740847
Random Forest	0.515904287977786	0.4030706229634485	0.7363493465419169
SVR	0.6625984948504582	0.5286864399395044	0.5650979378221219

Our goal was to create a model that could predict the performance of a track on spotify, and determine the metrics that most contribute to a song's success. In the above figure we present the performance of four of the models we tested, using RMSE, MAE, and R^2 as metrics. The Random Forest model outperformed all other models, scoring the best in in all three metrics. The superior performance of the nonlinear models suggests that nonlinear trends and interactions between features are some of the main contributors of streaming success. For example, a song with both high playlist inclusion and danceability could have compounded success. Linear models fail to capture feature relationships like these, leading to worse performance overall. The top 5 coefficients that we got from our Linear Regression model were $in\_spotify\_playlists$, $in\_apple\_playlists$, $in\_deezer\_playlists$, key_A, and $key\_D\#$.[3] The top 5 feature importances that we got from a tuned Random Forest were $num\_in\_spotify\_playlists$, $num\_in\_deezer\_playlists$, $num\_released\_day$, $num\_valence\_\%$, and $num\_in\_apple\_playlists$.[4]

4.2 Error Analysis and Diagnostics



Our Analysis of prediction errors showed that all models underestimate stream counts for very popular songs. This is to be expected, as viral hits are outliers, and it is very difficult to predict performance of these tracks with our given features. This could suggest that some aspects of a song that lead to it being a top viral hit are not fully captured by our features. Our features were focused around attention on streaming sites and the music in the song itself, features such as inclusion on social media and in films may have helped minimize error in predicting songs with very high stream counts. The top 3 artists with the highest prediction error when we use random forest are Ed Sheeran, Kendrick Lamar, and Labrinth.[5]



5 Conclusion: Impact, Limitations, and Future Work

Our work shows that Spotify song performance can be predicted fairly well from a set of platform exposure variables like playlist/chart counts and audio/metadata features, after log transforming the outcome to reduce skewness. Across the models we experimented with, nonlinear approaches were clearly more effective: our tuned Random Forest achieved the best test performance (RMSE 0.516, MAE 0.403, R^2 0.736), suggesting that streaming success depends on nonlinear patterns and feature interactions that linear models do not capture well. Consistent with this, the most predictive features were exposure-related (especially playlist counts across platforms) along with a smaller set of audio/date features (e.g., release day, valence). These results are potentially useful for artists trying to maximize engagement.

The following are some limitations. First, our errors are largest for viral tracks: all models tended to underpredict the highest-stream songs, suggesting that the factors behind extreme levels of engagement aren't captured fully by our data/the models we tried. Relatedly, many platform exposure variables (e.g., playlist placements) may be part of a feedback loop, successful songs get added to more playlists, so strong predictive power does not necessarily imply a causal effect. Second, multicollinearity is present among predictors (we observed mostly moderate VIF values, with at least one notably high value), which can make coefficient-based interpretation unstable in linear models even when prediction is acceptable. Finally, the dataset structure is largely cross-sectional and limited to observed metadata/audio features and platform counts, so it may not generalize perfectly to brand-new releases, different time periods, or songs whose success is driven by external shocks (e.g., TikTok trends, film/TV placement, or major publicity events).

There are several ways to extend this project. (1) Add missing drivers of virality by incorporating external attention signals (social media/TikTok metrics, YouTube views, radio play, sync placements) to reduce underprediction at the top end. (2) Model dynamics over time (e.g., early-week streams predicting long-run outcomes) with a panel/time-series setup, which would better reflect how exposure and streams co-evolve. (3) Address feedback loops by separating "intrinsic" song features from "exposure" features.

6 Appendix

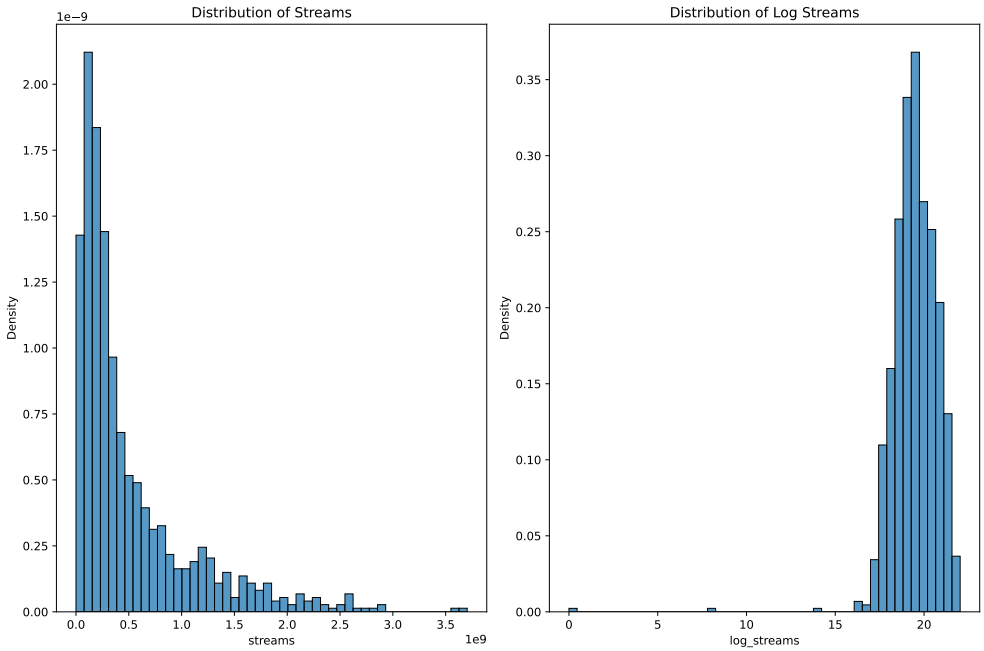


Figure 1: Streams vs Log Streams Distribution

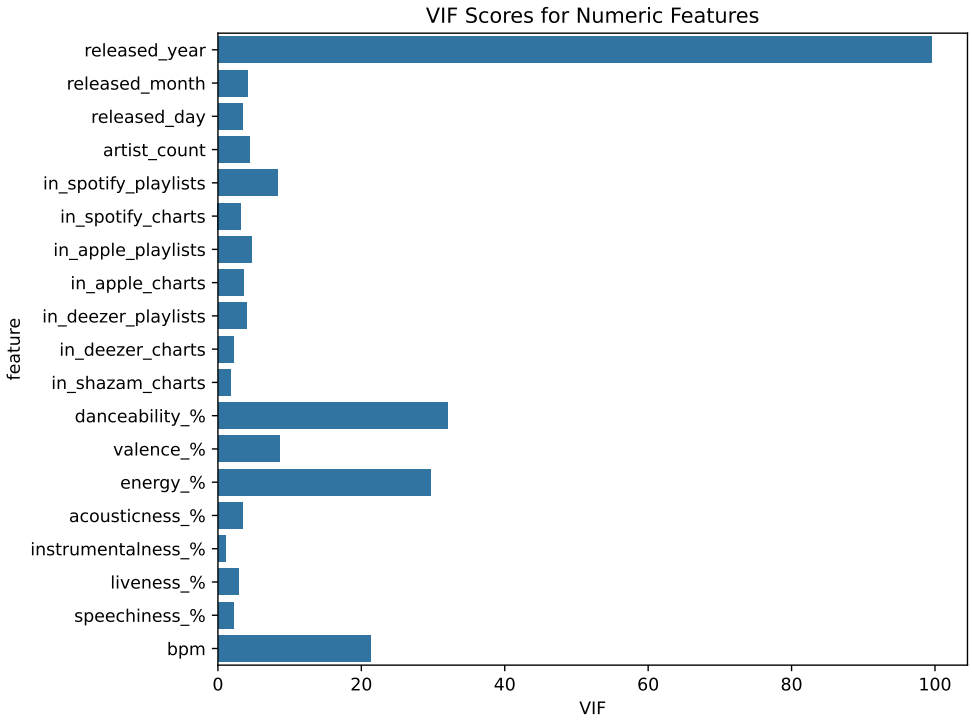


Figure 2: VIF Figure

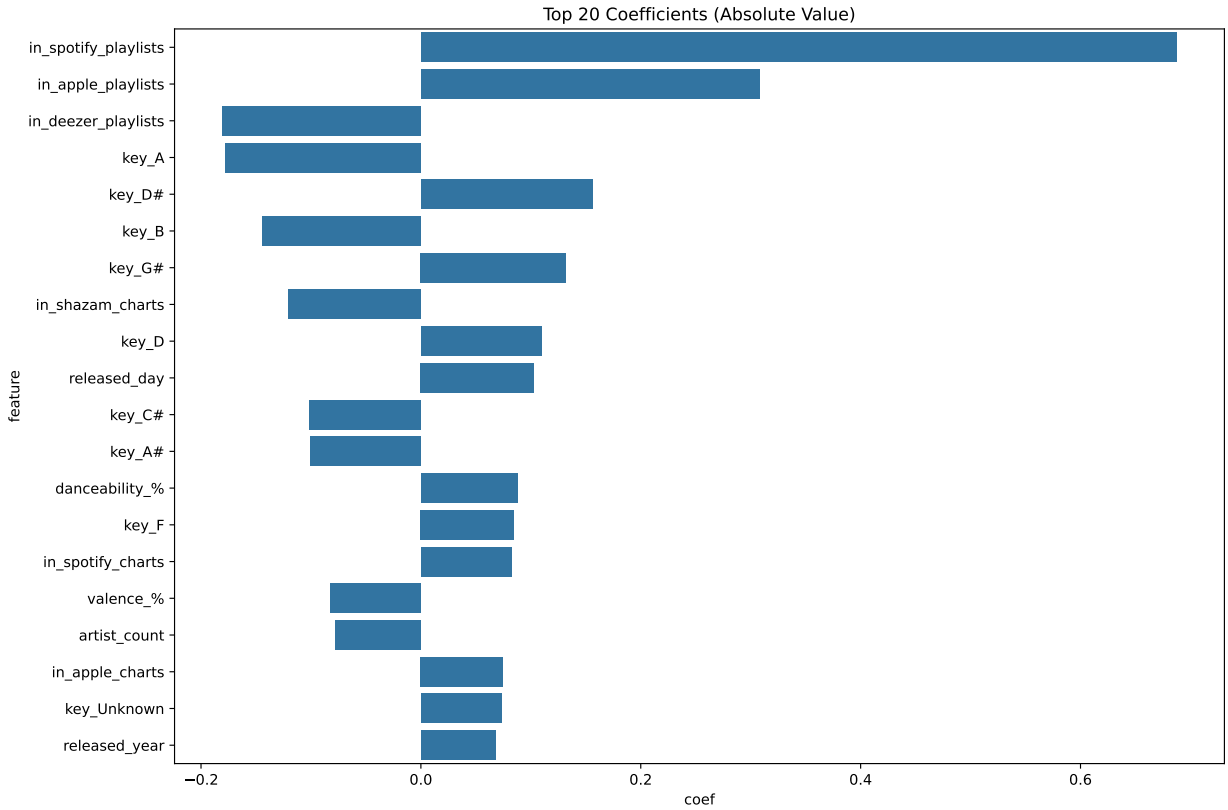


Figure 3: Linear Regression Coefficients

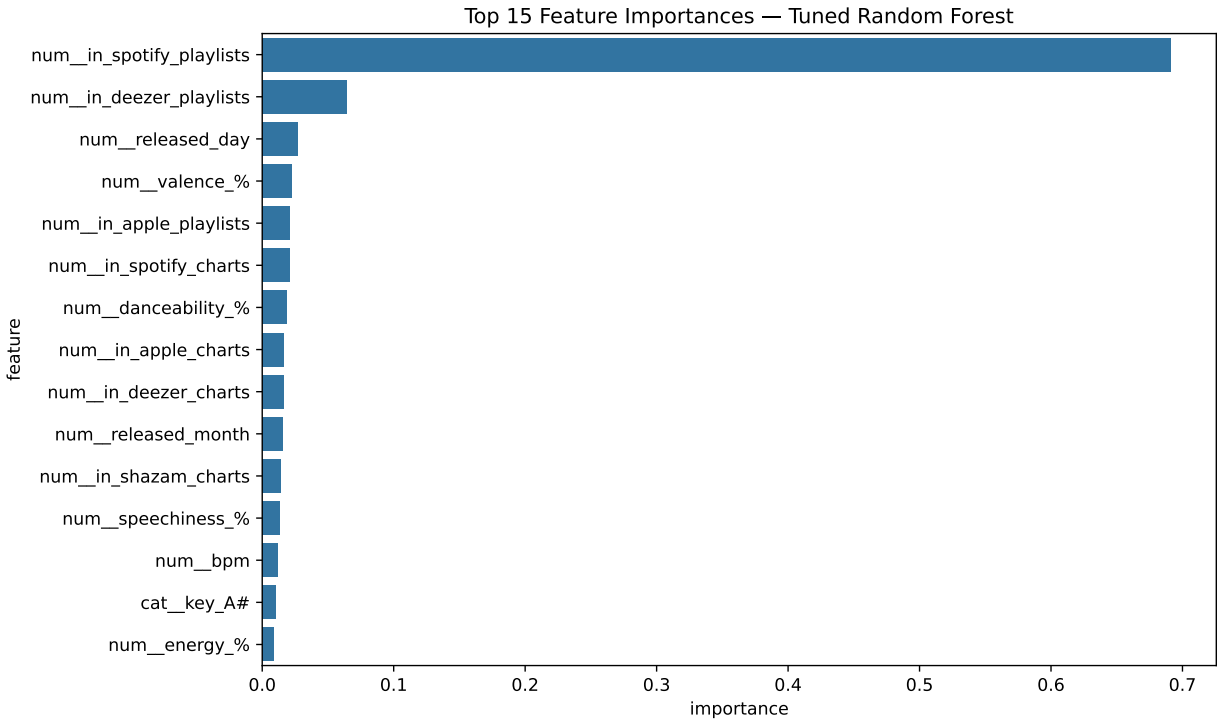


Figure 4: Random Forest Feature Importances

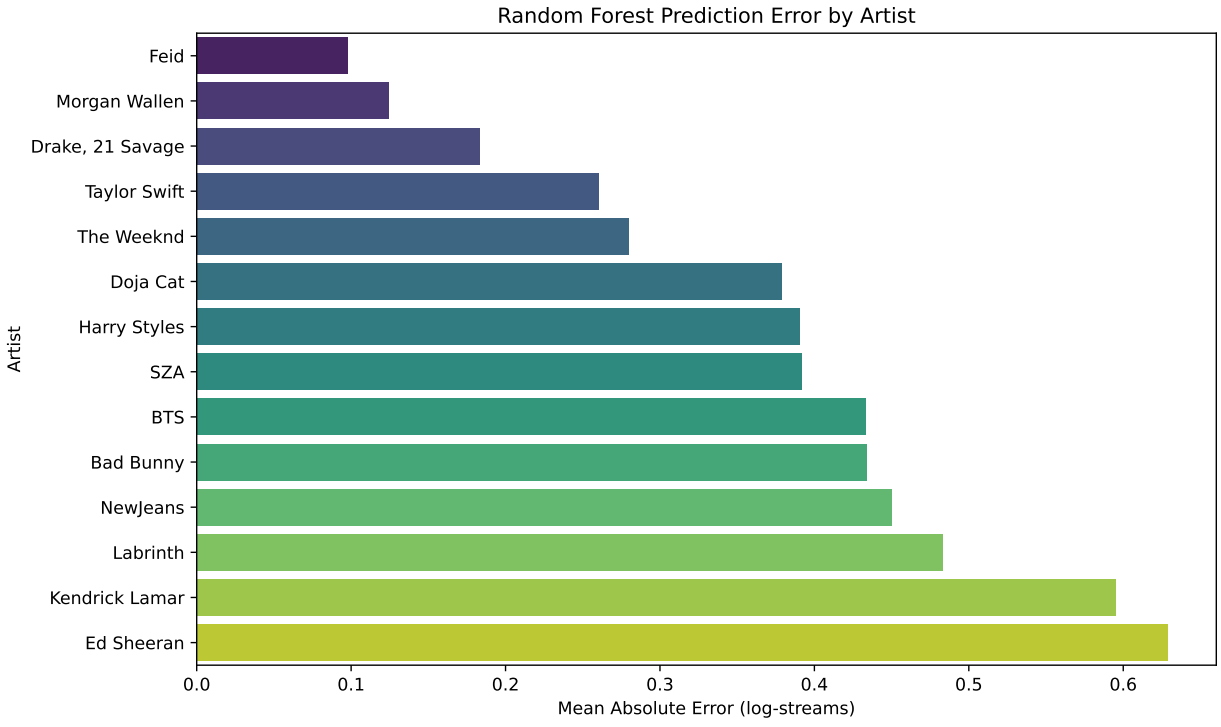


Figure 5: Artist MAE Barplot